

C++

Vojtěch Cimbura

Lecture II

04.11.2025

Quick Recap

- Last time:
 - Materials setup
 - **WIP** TetrisBlock Actor: C++ and BP
 - Setting material color, hiding / showing
- **TODO**
 - Finish TetrisBlock Actor C++
 - TetrisPlayer
 - Utilize Enhanced input
 - Create callbacks for input keybinds and verify functionality
 - TetrisGameMode
 - Board visualization
 - Delegate binding with Pawn
 - Implement game logic

TetrisBlock - Functions

- We'd like to have a basic cube mesh on the actor, assign the dynamic material to the mesh and be able to read and assign a mesh color
- **Constructor**
 - Load a default cube mesh from the engine
- **BeginPlay()**
 - Check if material is assigned
 - Create dynamic material instance and set the material to the mesh component
 - (Later) Set visibility of the actor to false
- Create Getter & Setter for **BlockColor** variable
 - Setter will also set material color - **SetVectorParameterValue()**
 - If the new color is **Black**, hide the actor
- Test the behavior by assigning a pseudorandom color to the block

Asserts in UE

- `check()` / `checkf()`

- Behaves like `assert` in C/C++
- Failed condition **stops execution**, shows dialog, breakpoint
- Does not evaluate the condition in optimized build

- `verify()` / `verifyf()`

- Similar to `check`, **stops execution**
- Makes sure that evaluated condition is not optimized out in optimized build configurations

- `ensure()` / `ensureMsgf()`

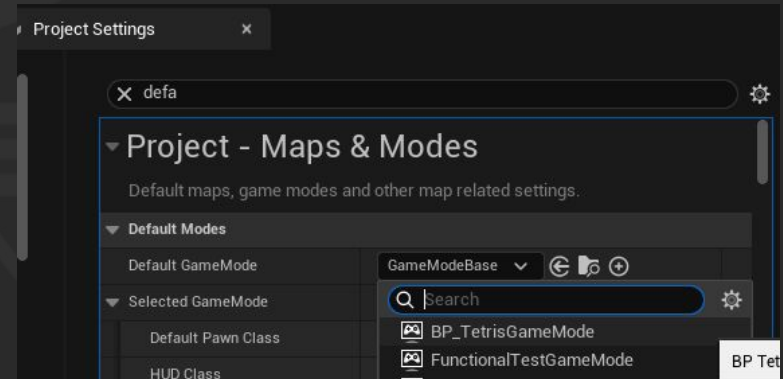
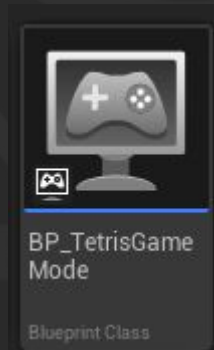
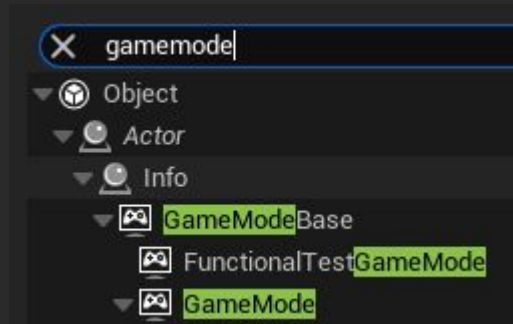
- Similar to `verify`, but **does not stop execution** -> non fatal errors
- Failure reported only once
 - Use `ensureAlways()` if you want to report always

Next steps

- We have the board building block set up - player input and the game logic left
- Game Mode
 - Spawn the board
 - (Later) Put all of our game logic in there
 - (Later) Player's Pawn has access to the game mode:
 - Direct access - quick implementation, but messy solution
 - Delegates - slower implementation, but preferred way
- Player input
 - Utilize Enhanced Input
 - Setup input actions
 - Create `TetrisGameMode` and set up our custom Pawn
 - Print out debug messages into the Output Log to verify functionality

Editor - Create and Set GameMode

- Create new C++ class:
 - **TetrisGameMode** (GameModeBase)
 - Game mode is a level persistent class accessible from everywhere
 - (For level transfer persistent class, use GameInstance)
- Create **BP_TetrisGameMode** based on the newly created C++ class
- Make the **BP_TetrisGameMode** the default game mode in Project Settings



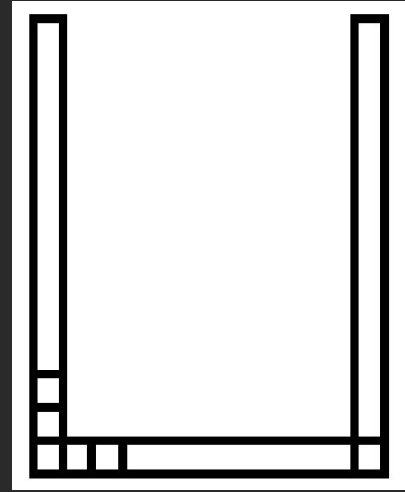
TetrisGameMode

- Variables:

- `TSubclassOf<ATetrisBlock> BlockClassToSpawn;`
- `TArray<ATetrisBlock*> Board;`

- Functions:

- `BeginPlay()`
 - Check if the variable `BlockClassToSpawn` is assigned
 - Spawn **16x20** (WxH) grid of `BP_TetrisBlock` actors
 - Block size = 100 Unreal Units (distance between them)
 - Store spawned actors in the `Board` array
 - After you verify blocks are spawned:
 - Set color of blocks to **White** unless the block is on a border, except the top row non-border blocks - then color is **Black**. When color is **Black**, hide the actor.



Editor - Create and Set TetrisPawn

- Create new C++ class:
 - **TetrisPlayer** (Pawn)
- Create **BP_TetrisPlayer** based on the newly created C++ class
 - Set it as Default Pawn in the **BP_TetrisGameMode**
- When you hit Play, you won't be able to move around anymore
 - Move **PlayerStart** and rotate it in a way that will fit the board view



Pawn

A Pawn is an actor that can be 'possessed' and receive input from a controller.



BP_TetrisPlayer

Blueprint Class

Editor - Enhanced Input

- Create & Navigate to a folder 'Input' in the Content Browser
- Create mapping context: **IMC_TetrisMappingContext**
- Create 3 Input Actions:
 - **IA_MoveBlock**
 - Axis1D (float)
 - Triggers: Pressed
 - **IA_PlaceBlock**
 - Digital (bool)
 - Triggers: Pressed + Released
 - **IA_RotateBlock**
 - Digital (bool)
 - Triggers: Pressed
- Map them to keys of your choice in the **IMC_TetrisMappingContext**
 - A key: Negate modifier to move block left

Player's Pawn - Variables

- Connection to the Enhanced Input
 - Add includes in **TetrisPlayer.h**:

```
#include "EnhancedInputComponent.h"  
#include "EnhancedInputSubsystems.h"  
#include "InputActionValue.h"
```

- Add public member variables to **TetrisPlayer.h**
 - Similarly for other **UInputActions**

```
public:  
    UPROPERTY(EditAnywhere)  
    Changed in 0 Blueprints  
    UInputMappingContext* TetrisMappingContext;  
  
    UPROPERTY(EditAnywhere)  
    Changed in 0 Blueprints  
    UInputAction* RotateBlock;
```

Learning material

- [UE Naming convention](#)
- [Programming in UE C++, the whole documentation](#)
- [Balancing BPs and C++](#)
- [Getting started with Unreal Engine](#)
- [Asserts in UE](#)
- [Objects in UE](#)

UNREAL
ENGINE